



VILNIUS TECH

FACULTY OF ELECTRONICS

DEPARTMENT OF COMPUTER SCIENCE
AND COMMUNICATIONS TECHNOLOGIES

Alexandre SAUVAN

KIKfu-21

Internet Technologies

Course project

Evaluate by Vytautas MAČAITIS

Vilnius, 2025

Contents

1	Introduction	4
1.1	What is a web site?	4
1.2	My work	4
2	Project description	5
2.1	Functional requirements	5
2.2	Non-Functional Requirements	5
3	Analysis and design	7
3.1	Use case diagram	7
3.2	Class diagram	8
3.3	Functional design	9
3.3.1	Website structure	9
3.3.2	Pages structure	10
3.4	Graphic design	11
3.4.1	Navigation system	11
3.4.2	Website appearance	11
4	Technical realization	12
4.1	Front-end	12
4.1.1	Structure (HTML)	13
4.1.2	Flexbox (CSS)	13
4.1.3	Animation (CSS)	13
4.1.4	Responsive design (CSS)	14
4.1.5	Display stocks (JS)	14
4.1.6	Verify form (JS)	14
4.2	Back-end	15
4.2.1	Navigation in the catalog (PHP)	15
4.2.2	Password hashing (PHP)	16
4.2.3	Contact form (PHP)	16
4.2.4	Catalog storage (XML)	17
4.2.5	User database (JSON)	17
4.3	Used tools	18
4.3.1	Integrated Development Environment (IDE)	18

<i>CONTENTS</i>	2
4.3.2 Validator HTML (W3C)	18
5 Conclusion	19
5.1 Result	19
5.2 Improvements planned	19

List of Figures

3.1	Use case diagram	8
3.2	Use case diagram	9
3.3	Page structure scheme	10
4.1	Front-end languages [2]	12
4.2	PHP back-end language [3]	15
4.3	Notepad++ logo [1]	18
4.4	Valid HTML5	18

Chapter 1

Introduction

1.1 What is a web site?

A website is a collection of web pages that are accessed through the Internet using a web browser. It can be as simple as a single page with text and images, or as complex as an interactive application. A website is made up of two main parts: the **front-end** and the **back-end**.

First, the front-end is created using HTML and CSS. These are not programming languages but markup and styling languages used to define the layout and visual appearance of the page.

Next, the back-end is built using other languages like PHP or Java. This part answers the question: "What happens behind the scenes?" Thanks to the backend, we can create dynamic pages and connect the website to a database.

1.2 My work

For this laboratory work, I chose to create a website for guitars sales! Because I love music and I played guitar for almost 10 years. But also because it's a good subject to many different languages with a system for logging in to a user account and filling out a basket to buy guitars.

Chapter 2

Project description

2.1 Functional requirements

The functional requirements describe the core features available on the guitar e-commerce website.

- A user can create an account using a username and password.
- A user can log in to their account securely.
- A user can log out of their account.
- A user can access the contact form to send a message to the support team.
- Only authenticated users can manage a shopping cart: add guitars, view their cart, update quantities, and remove items.
- An administrator has access to view the database containing users and carts, but cannot modify product listings or user data.

2.2 Non-Functional Requirements

The non-functional requirements define performance, usability, and security expectations for the website.

- The website must be entirely in English.
- Page load times must remain fast, even on average internet connections.
- The site must be compatible with all major web browsers (Chrome, Firefox, Edge, Safari).
- The user interface must be responsive and adapt to different screen sizes (desktop, tablet, and mobile).

- User passwords must be securely encrypted on the server using PHP.
- User data is stored in a JSON file, which must not be publicly accessible.
- All user inputs (e.g., login fields, contact form) must be validated both client-side and server-side to prevent malicious usage.
- The contact form must include basic validation to prevent spam or misuse.

Chapter 3

Analysis and design

3.1 Use case diagram

We aim to answer the following questions: Who are the users of the system? What are their interactions with it? This involves identifying the various actors as well as the use cases, i.e., the different features and functionalities offered by the system.

The main actors of the website are as follows:

- **Guest:** A visitor who does not have an account but can browse the website and view products.
- **Client:** A registered user who has an account and can purchase guitars and related products.
- **Administrator:** A person responsible for managing and maintaining the website (e.g., adding products, managing users, monitoring transactions).

Administrator is recognized by his username: `admin`. As the username is unique, nobody can create a new account claiming to have privileges reserved for administrators. The downside is that only one administrator is possible due to the uniqueness of the username. In the case of this website, it doesn't really matter, an administrator is just used to have a better view to resolve website bugs (direct access to the user database).

The main use cases associated with each of these actors are described in the use case diagram. Below is the use case diagram¹ that summarizes the key interactions between users and the system (figure 3.1).

¹Based on draw.io



Figure 3.1: Use case diagram

3.2 Class diagram

In this section, we'll look at the system's static entities. This is illustrated by the following class diagram² (figure 3.2):

There are just three objects because it's a little project, but the diagram is enough clear to understand how users, guitars and baskets interact together. Besides, no type are defined in the schema because it is not the priority for the website, the purpose of this diagram is to give an overview of the relationships between the various objects present.

²Based on StarUML

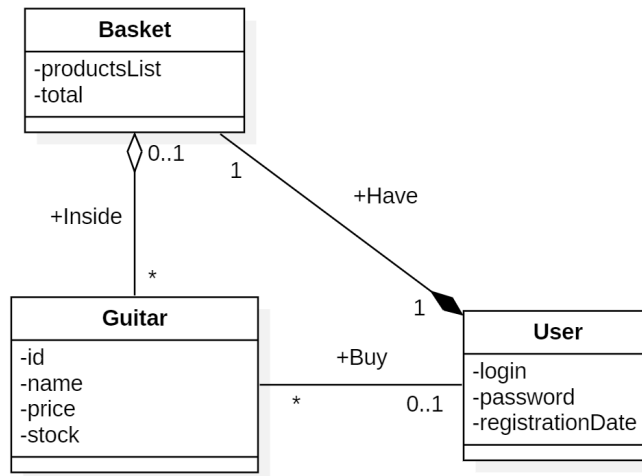


Figure 3.2: Use case diagram

3.3 Functional design

3.3.1 Website structure

The web site contains 7 pages.

Home The home page contains a presentation of the website with what the user can buy and a little part about the guitar history.

Categories Next, the categories page is used to display all guitars that a client can purchase. There are 3 categories for 3 kinds of guitar described below.

- **Classic:** characterized by its nylon strings.
- **Folk:** characterized by its metal strings.
- **Electric:** used a electric system with an amplifier.

My account This page is dedicated to the user's account. If not logged in, the page displays a login form. Otherwise, the customer's shopping cart is displayed.

Contact Finally, a contact page to allow customers to ask questions or request a refund on a purchased product, etc.

Sitemap To see all pages in the website and hierarchy between them.

Terms & Conditions Terms of Service

Privacy Notice Privacy Policy

3.3.2 Pages structure

All website pages have the same structure. Elements that change between pages are contained in the `<section>` of the page figure 3.3 shows the structure³.

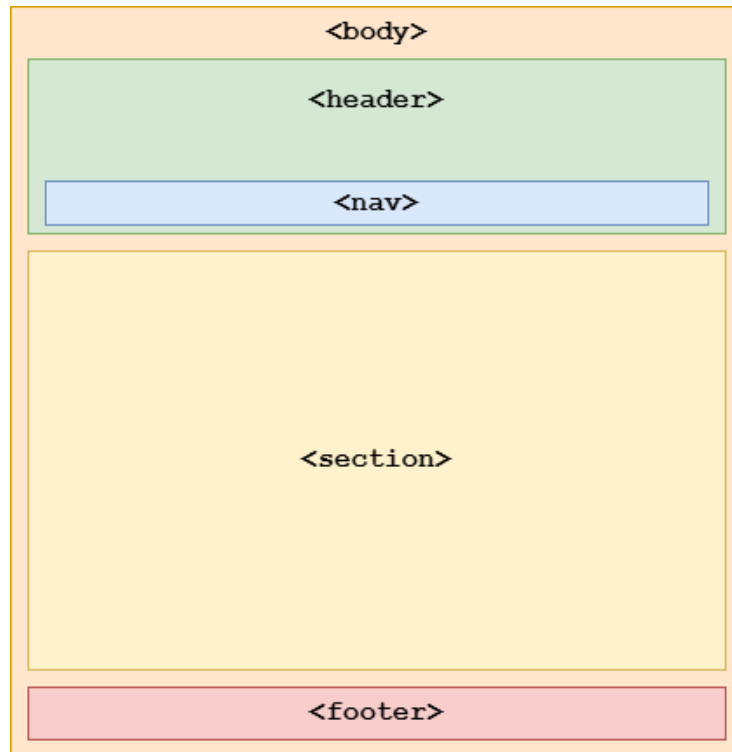


Figure 3.3: Page structure scheme

There are 3 part in the page that don't move.

- `<header>`: header of the page with the title and the navigation menu.
- `<nav>`: navigation menu with links to most important pages of the website.
- `<footer>`: contains links to consult the site map, terms and conditions, privacy notice, etc.

³Based on draw.io

3.4 Graphic design

3.4.1 Navigation system

A visual underline under the current page button lets visitors know which page they're on. This makes it easy for any user to know where they are and to choose another page to visit.

3.4.2 Website appearance

Each page should have its own title, so that the user knows without thinking which section of the site he's in. Simple sentences and short texts are preferred, as they make for quick reading for the user.

Chapter 4

Technical realization

4.1 Front-end

I chose **HTML 5** because it enables web pages to be structured more clearly and semantically, thanks to tags like `<header>`, `<section>` or `<footer>`, which make it easier for browsers, search engines and assistive technologies to understand the content.

Moreover, I used **CSS 3** to design a modern, attractive website, as it allows advanced management of the appearance and layout of HTML elements. It introduces numerous features such as animations, transitions, flexible grids (Flexbox) and media queries that facilitate the creation of adaptive (responsive) designs for different screens, especially mobiles.

Finally, **JavaScript** is an essential language for making websites interactive and dynamic. It enables you to manage user events (clicks, input, hovers, etc.) and modify page content without reloading (thanks to the DOM). JavaScript is also used to validate forms, create animations and shopping carts. Compatible with all modern browsers, it is at the heart of interactive websites.



Figure 4.1: Front-end languages [2]

4.1.1 Structure (HTML)

This is the web page skeleton:

```

1 <body>
2   <header>
3     <!-- Header + Navigation -->
4   </header>
5   <section>
6     <!-- Main part -->
7   </section>
8   <footer>
9     <!-- Other links -->
10  </footer>
11 </body>

```

Listing 4.1: HTML structure

4.1.2 Flexbox (CSS)

Flexbox is a concept introduced by CSS 3 to organize items in the HTML page. For example, I used this for the navigation menu to put links in `<ul>` in rows.

```

1 header nav ul
2 {
3   display: flex;
4   justify-content: center;
5   list-style-type: none;
6   padding: 0;
7   margin: 0;
8 }

```

Listing 4.2: Flexbox for the navigation menu

4.1.3 Animation (CSS)

On the page `catalogue.php`, I put a message for an offer. I added an animation to float the message on the page. I used `@keyframes` thanks to CSS 3:

```

1 @keyframes flotter {
2   0% {
3     transform: translateY(-2px);
4   }
5   50% {
6     transform: translateY(4px);
7   }
8   100% {
9     transform: translateY(-2px);
10  }
11 }

```

Listing 4.3: CSS animation

4.1.4 Responsive design (CSS)

For several years, the web is not only used through computers, but also TV or little devices like phone and tablets. That's why CSS 3 allows to add rules to change page CSS according to window size. My media queries used are for the header and footer if the window width is less than 600 pixels.

```
1 @media all and (max-width: 600px) {  
2   header nav ul {  
3     display: unset;  
4   }  
5   header nav ul li {  
6     margin: 10px;  
7   }  
8   footer div:first-child {  
9     display: unset;  
10    text-align: center;  
11  }  
12 }
```

Listing 4.4: Media queries

4.1.5 Display stocks (JS)

Thanks to JavaScript, I can add or remove a column in my table. In fact, via a button, users can display or not guitar inventory. JavaScript allows to change style of HTML tags.

```
1 let b = document.getElementById("affiche_stock");  
2  
3 b.onclick = function() {  
4   let e = document.getElementsByClassName("stock");  
5  
6   for (let i = 0 ; i < e.length ; i++) {  
7     if(e[i].style.display == "table-cell")  
8       e[i].style.display = "none";  
9     else  
10      e[i].style.display = "table-cell";  
11   }  
12 }
```

Listing 4.5: JavaScript code to display stocks

4.1.6 Verify form (JS)

A contact form has been added to enable users to send a message to support (product returns, questions, etc.). And for added convenience, JavaScript validation has been added to avoid any potential errors on the part of the user.

Of course, a back-end validation in PHP is mandatory to guarantee security website.

```

1 function erreurForm() {
2     e = false; // No error
3     // Last is <input type="submit"> (not a field)
4     for (let i = 0; i < document.contactForm.length-1; i++)
5     {
6         isValide = verify(document.contactForm[i]);
7         e = (e || !isValide);
8     }
9     return e;
10 }

```

Listing 4.6: JavaScript code to verify form data

4.2 Back-end

I chose **PHP** as back-end programming language for the website. Indeed, it is widely used to develop dynamic websites. It can be used to generate HTML content based on user actions, manage sessions and send forms. Widely documented and compatible with most hosting providers, PHP remains a reliable and efficient solution.



Figure 4.2: PHP back-end language [3]

To save data, JSON and XML are used to store users and guitars. JSON is lightweight and easy to handle with PHP, making it ideal for small projects. XML is used for structured data like the product catalog.

4.2.1 Navigation in the catalog (PHP)

All the catalog is in the `catalogue.php` page and this page is organized in categories (classic, folk and electric). A little navigation menu is created to choose the category.

```

1 <nav>
2     <h3>Guitar types</h3>
3     <p>To access the catalog, select a guitar type:</p>
4     <ul>
5         <li><a href="?cat=c">Classic</a></li>
6         <li><a href="?cat=f">Folk</a></li>
7         <li><a href="?cat=e">Electric</a></li>
8     </ul>
9 </nav>

```

Listing 4.7: Navigation with GET variables

Then, the PHP can display the category thanks to the GET variable. If the variable doesn't exist or has a bad value, the HTML table of content is displayed.

```

1 foreach ($_SESSION["catalogue"] as $id => $guitare) {
2     // Starts with the letter of the category
3     if (preg_match("#^" . $_GET["cat"] . "#", $id)) {
4         ?>
5         // HTML content...
6         <?php
7     }
8 }

```

Listing 4.8: Catalog display in PHP

4.2.2 Password hashing (PHP)

To ensure the security of your account, we recommend that you do not save the password directly in the database in clear text, but rather save a hash of the password. If a hacker succeeds in collecting data from your database, he won't be able to use the password. For that, PHP provides a function `password_hash` and you can choose your hashing algorithm, the easiest is to use `PASSWORD_DEFAULT`¹:

```

1 $pass_hash = password_hash($_POST["pass"], PASSWORD_DEFAULT);

```

Listing 4.9: Create a password hash in PHP

Next, to verify if a user put the good password in the log in form, we can use `password_verify` function to verify if the password corresponds to the hash stored in the database:

```

1 if (password_verify($_POST["pass"], $pass_hash))
2     // Connected
3 else
4     // Bad password

```

Listing 4.10: Verify that a password matches a hash in PHP

4.2.3 Contact form (PHP)

To contact the support or ask any questions, a form is available for everybody, users connected or not.

```

1 <form method="post" action="back/mail.php">
2     <!-- Fields -->
3 </form>

```

Listing 4.11: HTML contact form

When the submit button is pressed, the file `mail.php` is called and the script can send an e-mail thanks to this famous function is PHP:

¹Use the `bcrypt` algorithm (default as of PHP 5.5.0).

```
1 $e = !mail($mail, $sujet, $message, $header); // Send email
```

Listing 4.12: Mail function in PHP

The **header** is very important to hope to receive the e-mail. Indeed, some anti-spam software uses the match between the sender present in the FROM tag and the server from which the message is sent to detect a risk of fishing [4].

4.2.4 Catalog storage (XML)

To store guitars in the website catalog, a XML file is used. First XML allow to create our own tags. Thus, the first tag is to define the catalog, next categories with an attribute **type** to say it's witch category.

```
1 <catalogue>
2   <!-- 3 categories: classic, folk, electric -->
3   <categorie type="classique"></categorie>
4   <categorie type="folk"></categorie>
5   <categorie type="electrique"></categorie>
6 </catalogue>
```

Listing 4.13: Categories (empty) in the catalog in XML

Finally product with inside all tags to define the product like the ID, name, price and how many guitars are in stock.

```
1 <produit>
2   <id>c01</id>
3   <nom>Startone CG 851 1/8</nom>
4   <prix>38</prix>
5   <stock>11</stock>
6 </produit>
```

Listing 4.14: Example of product in XML

4.2.5 User database (JSON)

To manage users registered on the website, a JSON file is used as a database. Indeed, it is very practical to use with PHP to create a connection system. However, it's not a good choice if the security is asked because if a user find the path to the JSON file, he can consult all users in the database. For this website, it's not a problem because no confidential data are collected or stored.

```
1 {
2   "login": "admin",
3   "pass_hash": "$2y$10$kb.36d85\
IUQMY2DPdkMeuGh0F4eMLswBYVl5zdxMrTtvH9Blra.",
4   "date_inscription": "2024-02-21 07:01:54"
5 }
```

Listing 4.15: Example of user in JSON database

4.3 Used tools

4.3.1 Integrated Development Environment (IDE)

To create my website, I used my favorite IDE : **Notepad++**. Indeed, Notepad++ is a fast, lightweight editor that lets you open and edit files in an instant, even on less powerful computers. It offers syntax highlighting adapted to each language (HTML, CSS, JavaScript, PHP...), making code more readable. Thanks to its tabbed interface, it's easy to work on several files at the same time, and to navigate quickly between them. What's more, its advanced search tool lets you explore an entire project, with features such as regular expressions or sub-folder searches.

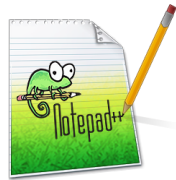


Figure 4.3: Notepad++ logo [1]

4.3.2 Validator HTML (W3C)

To verify if my HTML pages are correct, I used the W3C Markup Validation Service (<https://validator.w3.org/>) test my HTML code.



Figure 4.4: Valid HTML5

Chapter 5

Conclusion

5.1 Result

The guitar e-commerce website successfully meets the requirements defined at the start of the project. Users can register, log in, manage their shopping cart and contact the support team via a dedicated form. An administrator can securely access stored user and shopping cart data.

The platform guarantees good performance and user-friendliness thanks to a responsive design and fast loading times. Security measures such as password encryption and input validation have also been implemented to protect user data. Overall, the site provides a solid foundation for online shopping, combining clarity, efficiency and accessibility.

5.2 Improvements planned

Although the main features of the guitar e-commerce website have been successfully implemented, several improvements could be imagined.

In terms of functionality, the user experience could be enhanced by refining the shopping cart interface and implementing more robust feedback messages. A more advanced administration panel could also be added to help monitor site activity and view statistics.

Security could also be enhanced, particularly with regard to the protection of user data stored in JSON files. Additional techniques to prevent unauthorized access or manipulation of files would be welcome.

These improvements will make the platform more complete and ready for public use in a future release.

Bibliography

- [1] Motaz Alnuweiri. Icon for notepad++, 2010.
- [2] Bradley Nice. Amazingly useful html, css and javascript tools and libraries, 2016.
- [3] Colin Viebrock. Download logos and icons, 2001.
- [4] Site Du Zéro. Tutoriel : [e-mail] envoyer un e-mail en php, 2013.